МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Державний університет «Житомирська політехніка»

Факультет інформаційно-комп'ютерних технологій

Кафедра комп'ютерної інженерії

Дисципліна: Програмування

Спеціальність: 123 «Комп'ютерна інженерія»

ПОЯСНЮВАЛЬНА ЗАПИСКА

до лабораторної роботи № 2

Тема: Розгалужені та циклічні обчислювальні процеси у мові С, консольні меню та оформлення інженерної звітності

Варіант: № 1 (розрахований за номером у списку: (16 - 1) mod 15 + 1 = 1)

Виконав:

студент І курсу, групи КІ-26-1

Мошківський Даніїл Вадимович

Перевірив:

Лисенко Максим

Житомир — 2026

1. Мета роботи та індивідуальне завдання

Мета роботи: набути практичних навичок проектування комбінованих алгоритмів із використанням операторів розгалуження та вибору (if-else, switch-case), дослідити алгоритмічні особливості та практичну реалізацію циклів (for, while, do-while), опанувати чисельні методи наближеного обчислення функцій за допомогою степеневих рядів та рекурентних співвідношень, закріпити навички роботи з Git та оформлення інженерної документації.

Індивідуальне завдання (Варіант 1):

1. **Завдання 1 (Табулювання кусочно-неперервної функції):**

Реалізувати програму табулювання кусочно-неперервної функції Y(X) на проміжку [X_start, X_end] з кроком dX:

- $Y(X) = -X$, при $X \le 0$
- $Y(X) = X^2$, при $0 < X \le 2$
- $Y(X) = 4.0$, при $X > 2$

Програма має здійснювати валідацію вхідних даних (X_start < X_end, dX > 0) та виводити форматовану текстову таблицю.

2. **Завдання 2 (Наближене обчислення нескінченним рядом):**

Реалізувати програму наближеного обчислення функції e^X за допомогою розкладу в нескінченний степеневий ряд:

e^X = 1 + X/1! + X^2/2! + X^3/3! + ... = Σ (X^n / n!)

Область збіжності: -∞ < X < ∞. Обчислення кожного наступного члена здійснювати **виключно через рекурентне співвідношення**. Накопичення суми вести до досягнення точності |a_n| < ε з передбаченим апаратним захистом (Timeout Guard).

2. Математичний аналіз та рекурентні співвідношення

2.1. Аналіз Завдання 1

- **Область допустимих значень (ОДЗ):** Функция Y(X) складається з елементарних многочленів (-X, X^2, 4.0). Усі вирази визначені на всій числовій осі, ділення на нуль або добування квадратного кореня з від'ємного числа відсутні.

ОДЗ: X ∈ (-∞, +∞)

- **Аналіз точок стику:**
    - У точці X = 0: lim (-X) = 0, Y(0) = 0, lim X^2 = 0.
    - У точці X = 2: lim X^2 = 4, Y(2) = 4, lim 4.0 = 4.

Отже, функція є неперервною на всьому проміжку обчислення, точки розриву відсутні.

2.2. Аналітичний вивід рекурентного співвідношення для Завдання 2

Загальний n-й член степеневого ряду функції e^X має вигляд:

a_n = X^n / n!

Запишемо попередній (n-1)-й член ряду для n ≥ 1:

a_(n-1) = X^(n-1) / (n-1)!

Знайдемо коефіцієнт переходу R(n, X) = a_n / a_(n-1) шляхом скорочення факторіалів та степенів:

R(n, X) = (X^n / n!) / (X^(n-1) / (n-1)!) = (X^n / X^(n-1)) * ((n-1)! / n!) = X * ((n-1)! / (n * (n-1)!)) = X / n

Таким чином, рекурентна формула обчислення наступного доданка через попередній має вигляд:

a_0 = 1,

a_n = a_(n-1) * (X / n) (де n = 1, 2, 3, ...)

3. Опис алгоритмів та логічних рішень

Завдання 1 (task1.c):

1. **Введення та валідація:** Програма запитує змінні x_start, x_end, dx типу double. За допомогою умовного оператора if (x_start >= x_end || dx <= 0.0) перевіряється

коректність вхідних даних. У разі помилки виводиться повідомлення в потік stderr і програма завершується з кодом 1.

2. **Табулювання:** Використовується параметричний цикл for. Для запобігання втрати останньої точки через похибку округлення типів з плаваючою крапкою умова продовження циклу записана як x <= x_end + dx / 2.0.
3. **Обчислення значення:** Всередині циклу конструкція if-else if-else перевіряє, до якого з трьох інтервалів належить поточне значення X.
4. **Форматований вивід:** Результати виводяться у вигляді текстової таблиці із фіксованою шириною колонок (%10.2lf для аргументу та %12.4lf для значення).

Завдання 2 (task2.c):

1. **Введення та захист:** Перевіряється значення точності eps > 0.
2. **Накопичення суми:** Реалізовано цикл while (fabs(term) >= eps && iter < MAX_ITER).
3. **Оптимізація обчислень:** На кожній ітерації поточний доданок term перераховується за формулою term = term * (x / n), що повністю виключає використання важких бібліотечних функцій pow() та обчислення факторіалу.
4. **Timeout Guard:** Впроваджено лічильник ітерацій iter із константою MAX_ITER = 100000. Якщо ряд розбігається або досягнуто граничну кількість кроків, цикл примусово переривається, а користувач отримує відповідне попередження.

4. Лістинг вихідного коду

Лістинг task1.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    double x_start, x_end, dx;

    printf("Введіть X_start, X_end та dX: ");
    if (scanf("%lf %lf %lf", &x_start, &x_end, &dx) != 3) {
        fprintf(stderr, "Помилка: Некоректний формат введення даних.\n");
        return 1;
    }

    // Обов'язкова валідація вхідних параметрів
    if (x_start >= x_end || dx <= 0.0) {
        fprintf(stderr, "Помилка валідації: Очікується X_start < X_end та dX > 0.\n");
        return 1;
    }

    // Вивід текстової таблиці
    printf("+------------+--------------+\n");
    printf("|     X      |     Y(X)     |\n");
    printf("+------------+--------------+\n");

    for (double x = x_start; x <= x_end + dx / 2.0; x += dx) {
        double y;

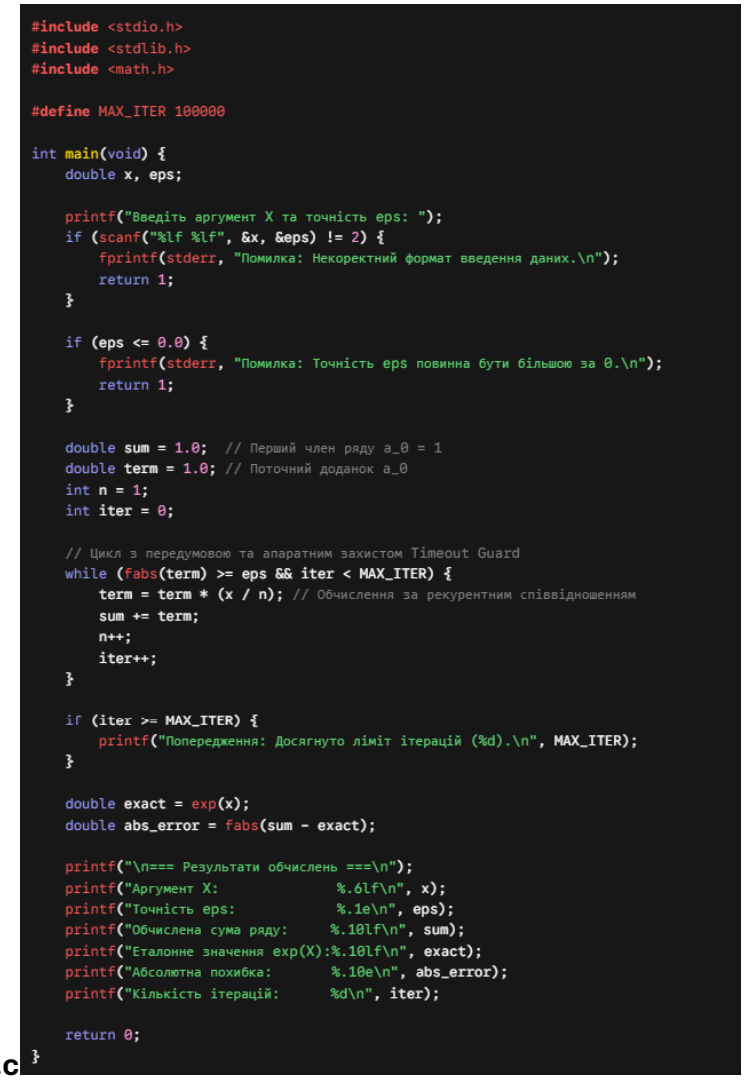
        if (x <= 0.0) {
            y = -x;
        } else if (x <= 2.0) {
            y = x * x;
        } else {
            y = 4.0;
        }

        printf("| %10.2lf | %12.4lf |\n", x, y);
    }

    printf("+------------+--------------+\n");

    return 0;
}
```

Лістинг task2.c



```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAX_ITER 100000

int main(void) {
    double x, eps;

    printf("Введіть аргумент X та точність eps: ");
    if (scanf("%lf %lf", &x, &eps) != 2) {
        fprintf(stderr, "Помилка: Некоректний формат введення даних.\n");
        return 1;
    }

    if (eps <= 0.0) {
        fprintf(stderr, "Помилка: Точність eps повинна бути більшою за 0.\n");
        return 1;
    }

    double sum = 1.0;  // Перший член ряду a_0 = 1
    double term = 1.0; // Поточний доданок a_0
    int n = 1;
    int iter = 0;

    // Цикл з передумовою та апаратним захистом Timeout Guard
    while (fabs(term) >= eps && iter < MAX_ITER) {
        term = term * (x / n); // Обчислення за рекурентним співвідношенням
        sum += term;
        n++;
        iter++;
    }

    if (iter >= MAX_ITER) {
        printf("Попередження: Досягнуто ліміт ітерацій (%d).\n", MAX_ITER);
    }

    double exact = exp(x);
    double abs_error = fabs(sum - exact);

    printf("\n=== Результати обчислень ===\n");
    printf("Аргумент X:              %.6lf\n", x);
    printf("Точність eps:            %.1e\n", eps);
    printf("Обчислена сума ряду:     %.10lf\n", sum);
    printf("Еталонне значення exp(X):%.10lf\n", exact);
    printf("Абсолютна похибка:       %.10e\n", abs_error);
    printf("Кількість ітерацій:      %d\n", iter);

    return 0;
}
```

5. Результати тестування та верифікація

5.1. Результати табулювання для Завдання 1

Вхідні дані: X_start = -1.0, X_end = 3.0, dX = 0.5.

X	Y(X)
-1.00	1.0000
-0.50	0.5000
0.00	0.0000
0.50	0.2500
1.00	1.0000
1.50	2.2500
2.00	4.0000
2.50	4.0000

| 3.00 | 4.0000 |

+----------+----------+

5.2. Зведена таблиця збіжності ряду для Завдання 2 (X = 2.0)

X	Точність ε	Обчислена сума	Еталонне значення exp(2.0)	Абсолютна похибка	Кількість ітерацій
2.0	10^{-2}	7.3809523810	7.3890560989	$8.1037 \cdot 10^{-3}$	6
2.0	10^{-4}	7.3890460197	7.3890560989	$1.0079 \cdot 10^{-5}$	9
2.0	10^{-6}	7.3890560521	7.3890560989	$4.6833 \cdot 10^{-8}$	11
2.0	10^{-8}	7.3890560962	7.3890560989	$2.7381 \cdot 10^{-9}$	12

5.3. Перевірка роботи захисних механізмів (Некоректні вхідні дані)

1. **Тест Task 1 при від'ємному кроці (dX = -0.5):**

Введіть X_start, X_end та dX: 1.0 5.0 -0.5

Помилка валідації: Очікується X_start < X_end та dX > 0.

2. **Тест Task 2 при некоректній точності (ε = -0.001):**

Plaintext

Введіть аргумент X та точність eps: 2.0 -0.001

Помилка: Точність eps повинна бути більшою за 0.

6. Висновки

1. **Оцінка швидкості збіжності:** Розклад e^X у ряд Маклорена має високу швидкість збіжності. Для досягнення точності ε = 10^{-8} при X = 2.0 знадобилося лише 12 ітерацій циклу.
2. **Ефективність рекурентного обчислення:** Використання рекурентного співвідношення a_n = a_(n-1) * (X / n) дозволило знизити обчислювальну складність кожної ітерації до двох арифметичних операцій (одне множення та одне ділення). Це усунуло потребу у виклику важких функцій pow() та обчисленні факторіалів, що унеможливило виникнення арифметичного переповнення типів даних.
3. **Досягнення мети:** Мета лабораторної роботи досягнута в повному обсязі: повністю реалізовано алгоритми табулювання кусочної функції та накопичення

суми нескінченного ряду, створено захисні механізми валідації даних, а код зафіксовано в системі Git за стандартом Conventional Commits.